

Reviewer Pack — Minimal Tropical Derivative Rank Correlation with Resolution...

Entry 2299785813fe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 12:00:02 UTC

Minimal Tropical Derivative Rank Correlation with Resolution Proof Width

Entry ID: 2299785813fe **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 12:00:02 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Tropical Derivatives) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Boolean formula φ , the minimal tropical derivative rank ($\text{mtr}(\varphi)$) of its associated Tseitin formula is linearly correlated with its resolution proof width $w(\varphi)$, such that $\text{mtr}(\varphi) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Tropical derivatives provide a way to analyze the complexity of functions over the tropical semiring, which might capture the essence of resolution proof construction in a non-commutative algebraic setting. This correlation could expose a new perspective on the structure of resolution proofs.

Taxonomy category: TROPICAL_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 145fa07af976c0d5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, the correlation coefficient between $\text{mtr}(\varphi)$ and $w(\varphi)$ must be ≥ 0.8 with a p-value ≤ 0.01 . To falsify it, any seed must exhibit a correlation coefficient < 0.5 or a p-value > 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND resolution proof complexity - Tseitin formula AND tropical derivative rank $\text{mtr}(\phi)$ - resolution proof width $\Theta(w(\phi))$ AND Tseitin formula

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = -A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] += factor * A[i][k]
            b[j] += factor * b[i]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def matrix_mult(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

```

def determinant(A, n):
    det = 0
    if n == 1:
        return A[0][0]
    elif n == 2:
        return A[0][0] * A[1][1] - A[0][1] * A[1][0]
    else:
        for c in range(n):
            det += ((-1) ** c) * A[0][c] * determinant([row[:c] + row[c+1:] for row in A[1:]], n-1)
    return det

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tseitin_formula(n, clause_distribution):
        literals = list(range(1, 2*n+1))
        clauses = []
        for i in range(1, n+1):
            clauses.append([i])
        for i in range(1, n+1):
            for j in range(i+1, n+1):
                clauses.append([-i, -j, 2*n+i])
                clauses.append([-i, -j, 2*n+j])
                clauses.append([i, j, -(2*n+i)])
                clauses.append([i, j, -(2*n+j)])
        for i in range(1, n+1):
            for clause in random.sample(clause_distribution[i], clause_distribution[i][0]):
                if clause < 0:
                    literals.append(-i)
                else:
                    literals.append(i)
                    clauses.append([clause])
        return literals, clauses

    def tropical_derivative_rank(literals, clauses, p):
        n = len(literals) // 2
        f_phi = [0] * (n + 1)
        for clause in clauses:
            term = 1
            for lit in clause:
                if lit > 0:
                    term *= literals[lit - 1]
                else:
                    term += literals[-lit - 1]
            f_phi[term % p] += 1
        J = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(1, n + 1):
            for j in range(1, n + 1):
                if i != j:
                    J[i][j] = literals[j - 1]
        rank = determinant(J, n)
        return rank

    def resolution_width(literals, clauses):
        # Simplified DPLL solver to estimate width
        def solve(lits_true, lits_false):

```

```

stack = []
while stack or lits_true:
    if not stack:
        stack.append(next((lit for lit in literals if lit not in lits_true and -lit not in lits_true)))
    lit = stack[-1]
    if lit is None:
        return len(lits_true)
    if lit > 0:
        lits_true.add(lit)
    else:
        lits_false.add(-lit)
    for clause in clauses:
        if all(lit not in clause and -lit not in clause for lit in literals):
            stack.pop()
            break
    return len(lits_true)

width = float('inf')
for i in range(1 << n):
    lits_true = set()
    lits_false = set()
    for j in range(n):
        if (i >> j) & 1:
            lits_true.add(j + 1)
        else:
            lits_false.add(-(j + 1))
    width = min(width, solve(lits_true, lits_false))
return width

n_values = [5, 10, 15, 20, 30, 40]
mtr_values = []
w_values = []

for n in n_values:
    clause_distribution = {i: random.randint(1, min(i, 3)) for i in range(1, n+1)}
    literals, clauses = tseitin_formula(n, clause_distribution)
    p = 101
    mtr = tropical_derivative_rank(literals, clauses, p)
    w = resolution_width(literals, clauses)
    mtr_values.append(mtr)
    w_values.append(w)

correlation_coefficient = sum((mtr_values[i] - mean_mtr) * (w_values[i] - mean_w) for i in range(len(mtr_values)))
mean_mtr = sum(mtr_values) / len(mtr_values)
mean_w = sum(w_values) / len(w_values)
p_value = 2 * (1 - abs(correlation_coefficient))

conjecture_holds = correlation_coefficient >= 0.8 and p_value <= 0.01
counterexample = "" if conjecture_holds else f"Correlation: {correlation_coefficient}, P-value: {p_value}"

return {
    "metric_name": "Tropical Derivative Rank vs Resolution Width",
    "metric_value": correlation_coefficient,
    "instances_tested": len(n_values),
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
}

```

```

        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    if not sys.argv[1:]:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
    else:
        seeds = [int(seed) for seed in sys.argv[1:]]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        print("RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8 or p-value > 0.05")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2af69lab.py", line 172, in <module>
    trial_result = run_trial(seed)
    ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2af69lab.py", line 139, in run_trial
    literals, clauses = tseitin_formula(n, clause_distribution)
    ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2af69lab.py", line 74, in tseitin_formula
    for clause in random.sample(clause_distribution[i], clause_distribution[i][0]):
    ~~~~~^~~~~~
TypeError: 'int' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the correlation coefficient and p-value. | next: Review the code for potential bugs that cause it to crash during execution.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23536
2	propose	ollama_remote	glm4:latest	0	0	13949
3	propose	ollama_remote	glm4:latest	0	0	12029
4	preregistration	ollama_remote	glm4:latest	0	0	9168
5	novelty	ollama_remote	glm4:latest	0	0	13076
6	novelty	ollama_remote	glm4:latest	0	0	15439
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17287
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28548
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24394
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35064
11	judge	ollama_remote	glm4:latest	0	0	13431

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 205922 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/2299785813fe.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2299785813fe.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2299785813fe.tar.gz` (if generated)